5

插叙：进程 API

补充：插叙

插叙这类章节讲的是系统中更偏实践的内容，尤其关注操作系统 API 以及怎么使用它们。如果你不喜欢实践性的东西，可以跳过这些插叙。不过你应该喜欢实践性的东西，因为，嗯，它们在现实生活中通常很有用；比如，公司一般不会因为你那些不实用的本事而雇用你。

在这个插叙中，我们讨论 UNIX 系统中的进程创建（process creation）。UNIX 用一对系统调用（system call）`fork()` 和 `exec()` 来创建新进程（process），这是最耐人寻味的进程创建方式之一。第三个例程 `wait()`，则供进程用来等待它创建的进程执行完毕。下面我们结合几个简单的例子，更详细地介绍这些接口。于是，我们的问题是：

关键问题：如何创建和控制进程

操作系统应该为创建和控制进程提供哪些接口？这些接口应该怎样设计，才能做到功能强大、容易使用、性能又高？

5.1 `fork()` 系统调用

`fork()` 系统调用用来创建新进程 [C63]。不过要事先提醒你：它肯定是你调用过的最奇怪的例程[1]。具体来说，假设你有一个正在运行的程序，它的代码如图 5.1 所示；仔细看看这段代码，最好自己敲一遍并运行一下！

运行这个程序（名为 p1.c）时，你会看到如下输出：

```
prompt> ./p1
hello (pid:29146)
parent of 29147 (pid:29146)
child (pid:29147)
prompt>
```

[1]好吧，我们承认这一点并没有十足的把握；谁知道你在没人看着的时候都调用了些什么例程呢？但无论你调用例程的习惯有多特别，`fork()` 都够古怪的。

```
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <unistd.h>
4
5  int main(int argc, char *argv[]) {
6      printf("hello (pid:%d)\n", (int) getpid());
7      int rc = fork();
8      if (rc < 0) {
9          // fork 失败
10         fprintf(stderr, "fork failed\n");
11         exit(1);
12     } else if (rc == 0) {
13         // 子进程（新进程）
14         printf("child (pid:%d)\n", (int) getpid());
15     } else {
16         // 父进程走这条路径（main）
17         printf("parent of %d (pid:%d)\n",
18                rc, (int) getpid());
19     }
20     return 0;
21 }
```

图 5.1：**调用 `fork()`（p1.c）**

我们来更详细地看看 p1.c 中发生了什么。进程刚开始运行时，会打印一条 hello 消息，消息里包含它的**进程标识符**（process identifier），也叫 **PID**。这个进程的 PID 是 29146；在 UNIX 系统中，如果想对某个进程做点什么，比如让它停止运行，就要用 PID 来指定这个进程。到目前为止一切都好。

接下来是有意思的部分。进程调用了 `fork()` 系统调用，这是操作系统提供的创建新进程的方式。奇怪之处在于：新创建的进程（几乎）是调用进程的一份精确副本。也就是说，在操作系统看来，现在好像有两个 p1 程序在运行，而且两者都即将从 `fork()` 系统调用返回。新创建的进程叫作**子进程**（child），与之相对，创建它的进程叫作**父进程**（parent）。子进程并不像你可能预想的那样从 `main()` 开始运行（注意，“hello”消息只打印了一次）；它就像自己调用了 `fork()` 一样，直接从那里开始了自己的生命。

你可能已经注意到：子进程并不是一模一样的副本。具体来说，虽然它现在有了自己的地址空间（address space）副本（即自己的私有内存）、自己的寄存器、自己的 PC 等等，但它从 `fork()` 返回给调用者的值是不同的。父进程收到的是新创建的子进程的 PID，而子进程收到的返回值是零。这种区别很有用，因为这样一来，就很容易写出分别处理这两种情况的代码（如上所示）。

你可能还注意到：（p1.c 的）输出并不是**确定的**（deterministic）。子进程创建之后，系统中就有了两个我们关心的活动进程：父进程和子进程。假设我们在单 CPU 的系统上运行（为简单起见），那么这时父进程和子进程都有可能先运行。在上面的例子中，是父进程先运行，所以它先打印出了消息。在其他情况下，也可能反过来，如下面这次输出所示：

```
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <unistd.h>
4  #include <sys/wait.h>
5
6  int main(int argc, char *argv[]) {
7      printf("hello (pid:%d)\n", (int) getpid());
8      int rc = fork();
9      if (rc < 0) {          // fork 失败，退出
10         fprintf(stderr, "fork failed\n");
11         exit(1);
12     } else if (rc == 0) { // 子进程（新进程）
13         printf("child (pid:%d)\n", (int) getpid());
14     } else {               // 父进程走这条路径
15         int rc_wait = wait(NULL);
16         printf("parent of %d (rc_wait:%d) (pid:%d)\n",
17                rc, rc_wait, (int) getpid());
18     }
19     return 0;
20 }
```

图 5.2：**调用 `fork()` 和 `wait()`（p2.c）**

```
prompt> ./p1
hello (pid:29146)
child (pid:29147)
parent of 29147 (pid:29146)
prompt>
```

CPU **调度程序**（scheduler）决定某一时刻运行哪个进程，我们很快就会详细讨论这个话题。由于调度程序很复杂，我们通常无法有把握地断定它会怎么选择，也就无法断定哪个进程会先运行。事实证明，这种**不确定性**（non-determinism）会带来一些有趣的问题，在**多线程程序**（multi-threaded programs）中尤其如此；因此，等到本书第二部分研究**并发**（concurrency）时，我们会看到多得多的不确定性。

5.2 `wait()` 系统调用

到目前为止，我们还没做多少事：只是创建了一个子进程，让它打印一条消息然后退出。有时候，让父进程等子进程把手头的事做完，其实相当有用。这项任务由 `wait()` 系统调用（或者它更完整的兄弟 `waitpid()`）完成；详见图 5.2。

在这个例子（p2.c）中，父进程调用 `wait()` 来阻塞（即推迟自己的执行），直到子进程执行完毕。子进程结束后，父进程不再阻塞，`wait()` 在父进程中返回。

在上面的代码中加入 `wait()` 调用，输出就变成确定的了。你能看出为什么吗？来，想一想。

（等你想一想……好了）

既然你已经想过了，下面是输出：

```
prompt> ./p2
hello (pid:29266)
child (pid:29267)
parent of 29267 (rc_wait:29267) (pid:29266)
prompt>
```

有了这段代码，我们就知道子进程总是先打印。为什么能知道呢？嗯，它可能像之前那样直接先运行，于是在父进程之前打印。不过，如果碰巧是父进程先运行，它会立即调用 `wait()`；这个系统调用要等子进程运行完并退出后才会返回[2]。因此，即使父进程先运行，它也会礼貌地等子进程运行结束，然后 `wait()` 返回，父进程再打印自己的消息。

5.3 最后是 `exec()` 系统调用

进程创建 API 中最后一个、也很重要的部分是 `exec()` 系统调用[3]。如果你想运行一个和调用程序不同的程序，这个系统调用就很有用。比如，在 p2.c 中调用 `fork()`，只有在你想继续运行同一个程序的副本时才有用。然而，你常常想运行另一个程序；`exec()` 做的正是这件事（图 5.3）。

在这个例子中，子进程调用 `execvp()` 来运行程序 wc，也就是统计字数的程序。实际上，它对源文件 p3.c 运行 wc，于是告诉我们这个文件里有多少行、多少个单词、多少个字节：

```
prompt> ./p3
hello (pid:29383)
child (pid:29384)
      29     107    1030 p3.c
parent of 29384 (rc_wait:29384) (pid:29383)
prompt>
```

`fork()` 系统调用很奇怪；它的同伙 `exec()` 也不怎么正常。它做的事情是：给定一个可执行程序的名字（例如 wc）和一些参数（例如 p3.c），它从这个可执行程序中**加载**（load）代码（和静态数据），用它覆盖当前的代码段（和当前的静态数据）；程序的堆、栈和内存空间的其他部分都会被重新初始化。然后操作系统直接运行这个程序，把所有参数作为该进程的 argv 传进去。因此，它并没有创建新进程，而是把当前正在运行的程序（原来是 p3）变成了另一个正在运行的程序（wc）。子进程执行 `exec()` 之后，就几乎像 p3.c 从来没有运行过一样；对 `exec()` 的成功调用永远不会返回。

5.4 为什么？API 的设计动机

当然，你可能会有一个大大的疑问：创建新进程本该是件简单的事，为什么要设计出这么古怪的接口？事实证明，`fork()` 和 `exec()` 的分离对于构建 UNIX shell 至关重要，因

[2]也有少数情况下，`wait()` 会在子进程退出之前返回；和往常一样，详情请读手册页（manual page）。另外，对于本书中任何绝对化、不加限定的说法，比如“子进程总是先打印”或者“UNIX 是世界上最好的东西，比冰淇淋还好”，都要多加小心。

[3]在 Linux 上，`exec()` 有六个变体：`execl()`、`execlp()`、`execle()`、`execv()`、`execvp()` 和 `execvpe()`。详情请读手册页。

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <string.h>
#include <sys/wait.h>

int main(int argc, char *argv[]) {
    printf("hello (pid:%d)\n", (int) getpid());
    int rc = fork();
    if (rc < 0) {          // fork 失败，退出
        fprintf(stderr, "fork failed\n");
        exit(1);
    } else if (rc == 0) { // 子进程（新进程）
        printf("child (pid:%d)\n", (int) getpid());
        char *myargs[3];
        myargs[0] = strdup("wc");   // 程序："wc"
        myargs[1] = strdup("p3.c"); // 参数：输入文件
        myargs[2] = NULL;           // 标记数组结束
        execvp(myargs[0], myargs);  // 运行 wc 统计字数
        printf("this shouldn't print out");
    } else {               // 父进程走这条路径
        int rc_wait = wait(NULL);
        printf("parent of %d (rc_wait:%d) (pid:%d)\n",
               rc, rc_wait, (int) getpid());
    }
    return 0;
}
```

图 5.3：**调用 `fork()`、`wait()` 和 `exec()`（p3.c）**

为它让 shell 可以在调用 `fork()` 之后、调用 `exec()` 之前运行一段代码；这段代码可以修改即将运行的程序的环境，从而让各种有意思的功能都能轻松实现。

shell 只是一个用户程序[4]。它向你显示一个**提示符**（prompt），然后等你输入。你输入一条命令（即一个可执行程序的名字，外加若干参数）；在大多数情况下，shell 接着会找出这个可执行程序在文件系统（file system）中的位置，调用 `fork()` 创建一个新的子进程来

提示：做对（Lampson 定律）

正如 Lampson 在他广受好评的《Hints for Computer Systems Design》[L83] 中所说："**做对**（get it right）。抽象（abstraction）和简洁都不能代替做对。"有时候，你就是得做正确的事；而一旦做对了，它会比其他选择好得多。设计进程创建 API 的方法有很多；然而，`fork()` 和 `exec()` 的组合既简单又极其强大。在这一点上，UNIX 的设计者就是做对了。因为 Lampson 常常"做对"，所以我们用他的名字来命名这条定律。

[4]shell 有很多种，比如 tcsh、bash 和 zsh。你应该挑一个，读读它的手册页，深入了解它；所有 UNIX 专家都是这么做的。

运行这条命令，调用 exec() 的某个变体来运行命令，然后调用 wait() 等待命令完成。子进程完成后，shell 从 wait() 返回，再次打印出提示符，准备接收你的下一条命令。

fork() 和 exec() 的分离，让 shell 可以相当轻松地做一大堆有用的事。例如：

```
prompt> wc p3.c > newfile.txt
```

在上面的例子中，程序 wc 的输出被**重定向**（redirected）到输出文件 newfile.txt 中（大于号表示这种重定向）。shell 完成这项任务的方式相当简单：子进程创建之后、调用 exec() 之前，shell（准确地说，是在子进程中执行的代码）关闭**标准输出**（standard output），然后打开文件 newfile.txt。这样一来，即将运行的程序 wc 的所有输出都会被发送到这个文件，而不是屏幕上（打开的文件描述符（file descriptor）在 exec() 调用前后保持打开，所以能做到这一点 [SR05]）。

图 5.4（第 7 页）展示了一个正是这样做的程序。这种重定向之所以可行，依赖于对操作系统如何管理文件描述符的一个假设。具体来说，UNIX 系统从零开始查找空闲的文件描述符。在这个例子中，STDOUT_FILENO 会是第一个可用的描述符，因此在调用 open() 时被分配出去。之后子进程对标准输出文件描述符的写入，比如通过 printf() 这样的例程，就会被透明地导向新打开的文件，而不是屏幕。

运行 p4.c 程序的输出如下：

```
prompt> ./p4
prompt> cat p4.output
      32     109     846 p4.c
prompt>
```

关于这个输出，你会注意到（至少）两个有意思的细节。第一，运行 p4 时，看起来好像什么都没发生；shell 只是打印出命令提示符，马上就准备好接收你的下一条命令了。但事实并非如此：程序 p4 确实调用了 fork() 来创建新的子进程，然后通过调用 execvp() 运行了 wc 程序。屏幕上之所以看不到任何输出，是因为输出已经被重定向到文件 p4.output 中了。第二，你可以看到，当我们用 cat 查看这个输出文件时，运行 wc 应该得到的输出都在里面。很酷，对吧？

UNIX 的管道（pipe）也是用类似的方式实现的，不过用的是 pipe() 系统调用。在这种情况下，一个进程的输出被连接到内核中的一个**管道**（pipe），也就是一个队列（queue），另一个进程的输入也被连接到同一个管道；这样，一个进程的输出就能无缝地成为下一个进程的输入，从而可以把很长、很有用的一串命令连在一起。举个简单的例子：在文件中查找某个单词，再数一数这个单词出现了多少次；有了管道以及 grep 和 wc 这两个工具，这件事很简单：只要在命令提示符下输入 grep -o foo file | wc -l，然后惊叹于结果就行了。

最后，虽然我们只是从较高的层面勾勒了进程 API（process API），但关于这些调用，还有大量细节值得学习和消化；比如，等到本书第三部分讨论文件系统时，我们会进一步了解文件描述符。现在只需知道：fork()/exec() 组合是创建和操作进程的一种强大方式。

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <string.h>
#include <fcntl.h>
#include <sys/stat.h>
#include <sys/wait.h>

int main(int argc, char *argv[]) {
    int rc = fork();
    if (rc < 0) {
        // fork 失败
        fprintf(stderr, "fork failed\n");
        exit(1);
    } else if (rc == 0) {
        // 子进程：把标准输出重定向到文件
        close(STDOUT_FILENO);
        open("./p4.output", O_CREAT|O_WRONLY|O_TRUNC,
             S_IRWXU);
        // 现在 exec "wc"……
        char *myargs[3];
        myargs[0] = strdup("wc");   // 程序：wc
        myargs[1] = strdup("p4.c"); // 参数：要统计的文件
        myargs[2] = NULL;           // 标记数组结束
        execvp(myargs[0], myargs);  // 运行 wc 统计字数
    } else {
        // 父进程走这条路径（main）
        int rc_wait = wait(NULL);
    }
    return 0;
}
```

图 5.4：**以上全部再加上重定向（p4.c）**

5.5 进程控制与用户

除了 `fork()`、`exec()` 和 `wait()`，UNIX 系统中还有许多其他与进程交互的接口。例如，`kill()` 系统调用用来向进程发送**信号**（signals），包括让进程暂停、终止以及其他有用的指令。为了方便，大多数 UNIX shell 都把某些组合键配置成向当前正在运行的进程发送特定的信号；例如，control-c 向进程发送 SIGINT（中断，interrupt）信号，通常会终止它；control-z 发送 SIGTSTP（停止，stop）信号，让进程在执行中途暂停（之后可以用命令让它恢复运行，比如很多 shell 都有的内置命令 fg）。

整个信号子系统提供了一套丰富的基础设施，用来把外部事件传递给进程，包括在单个进程中接收和处理这些信号的方式，以及向单个进程乃至整个**进程组**（process groups）发送信号的方式。要使用这种通信方式，进程应该用 `signal()` 系统调用来“捕获”各种信号；这样做可以确保，当某个信号传递给进程时，进程会暂停正常执行，转而运行一段特定的代码来响应这个信号。想进一步了解信号及其诸多复杂细节，请读别的资料 [SR05]。

 © 2008–25, ARPACI-DUSSEAU

补充：RTFM——去读手册页

在本书中，提到某个系统调用或库调用时，我们常常会让你去读**手册页**（manual pages，简称 **man pages**）。手册页是 UNIX 系统上最初的文档形式；要知道，它们出现的时候，所谓的**万维网**（web）还不存在呢。

花些时间读手册页，是系统程序员成长过程中的关键一步；这些页面里藏着大量有用的小知识。特别值得一读的，是你所用的 shell（比如 **tcsh** 或 **bash**）的手册页，当然还有你的程序用到的所有系统调用的手册页（看看有哪些返回值和错误情况）。

最后，读手册页还能帮你免去一些尴尬。当你向同事请教 `fork()` 的某个复杂细节时，他们可能只会回一句："RTFM。"这是同事在委婉地催你去读手册页（Read The **Man pages**）。RTFM 里的 F 只是给这句话增添了一点色彩……

这自然引出一个问题：谁能向进程发送信号，谁不能？一般来说，我们所用的系统可能同时有很多人在用；如果其中某个人能随意发送 SIGINT 这样的信号去中断某个进程，很可能就此把它终止，系统的易用性和安全性就会受到损害。因此，现代系统对**用户**（user）这一概念有很明确的定义。用户先输入密码以建立凭证（credential），然后登录，获得对系统资源的访问权限。之后，用户可以启动一个或多个进程，并完全控制它们（暂停它们、杀掉它们等）。用户一般只能控制自己的进程；操作系统的职责是把资源（比如 CPU、内存和磁盘）分配给每个用户及其进程，以实现系统的整体目标。

5.6 实用工具

还有许多命令行工具也很有用。例如，用 ps 命令可以看到哪些进程正在运行；读一读手册页，了解可以传给 ps 的一些有用的选项。top 工具也很有用，它会显示系统中的进程，以及它们各自占用了多少 CPU 和其他资源。有趣的是，很多时候你运行 top，它都会声称自己是头号资源大户；看来它多少有点自恋。kill 命令可以向进程发送任意信号，稍微好用一点的 killall 也可以。使用这些命令一定要小心；如果你不小心杀掉了窗口管理器，你面前的这台计算机可能就变得很难用了。

补充：超级用户（root）

系统通常需要一个能**管理**（administer）系统的用户，这个用户不像大多数用户那样受到限制。这样的用户应该能杀掉任意进程，比如某个正以某种方式滥用系统的进程，即使这个进程不是由这个用户启动的。这样的用户还应该能运行一些强大的命令，比如 shutdown（不出所料，它会关闭系统）。在基于 UNIX 的系统中，这些特殊能力属于**超级用户**（superuser，有时也叫 **root**）。大多数用户不能杀掉其他用户的进程，但超级用户可以。当 root 很像当蜘蛛侠：能力越大，责任越大 [QI15]。因此，为了提高**安全性**（security），也为了避免代价高昂的错误，通常最好以普通用户的身份工作；如果确实需要以 root 身份工作，务必小心行事，因为计算世界里所有的破坏力现在都握在你手里了。

最后，还有许多不同种类的 CPU 监视工具，能让你快速了解系统的负载情况；例如，我们总是在 Macintosh 的工具栏上运行着 **MenuMeters**（Raging Menace 公司的软件），这样随时都能看到 CPU 的使用情况。一般来说，对正在发生的事了解得越多越好。

5.7 小结

我们介绍了一些与 UNIX 进程创建有关的 API：`fork()`、`exec()` 和 `wait()`。不过，我们只是触及了皮毛。想了解更多细节，当然要读 Stevens 和 Rago 的书 [SR05]，尤其是其中关于进程控制（process control）、进程关系和信号的几章；那里面的智慧值得好好汲取。

虽然我们对 UNIX 进程 API（process API）依然热情不减，但也应该指出，并不是所有人都这么看好它。例如，来自微软、波士顿大学和瑞士苏黎世联邦理工学院（ETH）的系统研究人员最近发表了一篇论文，详细阐述了 `fork()` 的一些问题，并提倡使用其他更简单的进程创建 API，比如 **`spawn()`** [B+19]。读一读这篇论文以及它引用的相关工作，了解这种不同的视角。虽然一般来说信任这本书是好事，但也要记住，作者是有自己观点的；这些观点未必（总是）像你以为的那样被广泛认同。

补充：进程 API 关键术语

- 每个进程都有一个名字；在大多数系统中，这个名字是一个数字，叫作**进程 ID**（process ID，**PID**）。
- 在 UNIX 系统中，**`fork()`** 系统调用用来创建新进程。创建者叫作**父进程**（parent process）；新创建的进程叫作**子进程**（child process）。就像现实生活中有时会发生的那样 [J16]，子进程和父进程几乎一模一样。
- **`wait()`** 系统调用让父进程可以等待子进程执行完毕。
- **`exec()`** 系列系统调用让子进程摆脱与父进程的相似，去执行一个全新的程序。
- UNIX **shell** 通常用 `fork()`、`wait()` 和 `exec()` 来启动用户命令；`fork` 和 `exec` 的分离使得**输入/输出重定向**（input/output redirection）、**管道**（pipe）等很酷的功能成为可能，而且完全不需要修改被运行的程序。
- 进程控制（process control）以**信号**（signal）的形式提供，信号可以让作业停止、继续，甚至终止。
- 某个人能控制哪些进程，由**用户**（user）这一概念来界定；操作系统允许多个用户进入系统，并确保每个用户只能控制自己的进程。
- **超级用户**（superuser）可以控制所有进程，其实还能做很多别的事；出于安全考虑，应该尽量少用、谨慎使用这个角色。

参考文献

[B+19] "A fork() in the road" by Andrew Baumann, Jonathan Appavoo, Orran Krieger, Timothy Roscoe. HotOS '19, Bertinoro, Italy. *A fun paper full of `fork()`ing rage. Read it to get an opposing viewpoint on the UNIX process API. Presented at the always lively HotOS workshop, where systems researchers go to present extreme opinions in the hopes of pushing the community in new directions.*

[C63] "A Multiprocessor System Design" by Melvin E. Conway. AFIPS '63 Fall Joint Computer Conference, New York, USA 1963. *An early paper on how to design multiprocessing systems; may be the first place the term `fork()` was used in the discussion of spawning new processes.*

[DV66] "Programming Semantics for Multiprogrammed Computations" by Jack B. Dennis and Earl C. Van Horn. Communications of the ACM, Volume 9, Number 3, March 1966. *A classic paper that outlines the basics of multiprogrammed computer systems. Undoubtedly had great influence on Project MAC, Multics, and eventually UNIX.*

[J16] "They could be twins!" by Phoebe Jackson-Edwards. The Daily Mail. March 1, 2016.. *This hard-hitting piece of journalism shows a bunch of weirdly similar child/parent photos and is frankly kind of mesmerizing. Go ahead, waste two minutes of your life and check it out. But don't forget to come back here! This, in a microcosm, is the danger of surfing the web.*

[L83] "Hints for Computer Systems Design" by Butler Lampson. ACM Operating Systems Review, Volume 15:5, October 1983. *Lampson's famous hints on how to design computer systems. You should read it at some point in your life, and probably at many points in your life.*

[QI15] "With Great Power Comes Great Responsibility" by The Quote Investigator. Available: https://quoteinvestigator.com/2015/07/23/great-power. *The quote investigator concludes that the earliest mention of this concept is 1793, in a collection of decrees made at the French National Convention. The specific quote: "Ils doivent envisager qu'une grande responsabilité est la suite inséparable d'un grand pouvoir", which roughly translates to "They must consider that great responsibility follows inseparably from great power." Only in 1962 did the following words appear in Spider-Man: "…with great power there must also come–great responsibility!" So it looks like the French Revolution gets credit for this one, not Stan Lee. Sorry, Stan.*

[SR05] "Advanced Programming in the UNIX Environment" by W. Richard Stevens, Stephen A. Rago. Addison-Wesley, 2005. *All nuances and subtleties of using UNIX APIs are found herein. Buy this book! Read it! And most importantly,* ***live it.***

 © 2008–25, ARPACI-DUSSEAU

作业（模拟）

本次模拟作业围绕 `fork.py` 展开，这是一个简单的进程创建模拟器，可以展示进程之间如何在一棵“家族”树中相互关联。关于如何运行这个模拟器，请阅读相应的 README。

问题

1. 运行 `./fork.py -s 10`，看看执行了哪些操作。你能预测每一步的进程树（process tree）是什么样子吗？用 `-c` 选项检查你的答案。试试不同的随机种子（`-s`），或者加入更多操作（`-a`），熟悉一下这个模拟器。
2. 模拟器提供的一项控制是 fork_percentage，由 `-f` 选项控制。它越高，下一个操作越可能是 fork；越低，下一个操作越可能是 exit。用大量操作（例如 `-a 100`）运行模拟器，把 fork_percentage 从 0.1 调到 0.9。随着这个比例的变化，你认为最终的进程树会是什么样子？用 `-c` 检查你的答案。
3. 现在用 `-t` 选项切换输出方式（例如运行 `./fork.py -t`）。给定一组进程树，你能说出执行了哪些操作吗？
4. 值得注意的一件事，是子进程退出时会发生什么：它自己的子进程在进程树中会怎样？为了研究这个问题，我们来构造一个具体的例子：

   ```
   ./fork.py -A a+b,b+c,c+d,c+e,c-
   ```

   在这个例子中，进程“a”创建“b”，“b”又创建“c”，“c”再创建“d”和“e”。然而，随后“c”退出了。你认为退出之后进程树应该是什么样子？如果使用 `-R` 选项呢？自己去了解一下孤儿进程（orphan process）会发生什么，补充一些背景知识。
5. 最后一个要探索的选项是 `-F`，它会跳过中间步骤，只要求你填出最终的进程树。运行 `./fork.py -F`，看看你能否通过观察生成的一系列操作写出最终的树。用不同的随机种子多试几次。
6. 最后，把 `-t` 和 `-F` 一起使用。这会显示最终的进程树，然后让你填出执行过的操作。通过观察这棵树，你能确定到底执行了哪些操作吗？哪些情况下能确定？哪些情况下不能？试试不同的随机种子，深入研究这个问题。

补充：编程作业

编程作业是一些小练习：你要编写代码，在真实的机器上运行，从而积累一些使用基本操作系统 API 的经验。毕竟，你（很可能）是个计算机科学家，所以应该喜欢写代码，对吧？如果不喜欢，你总还可以去搞计算机理论，不过那可相当难。当然，要真正成为专家，你得花不少时间在机器上折腾；事实上，要找一切借口写点代码，看看它是怎么工作的。花些时间，成为你知道自己能成为的那位大师。

 © 2008–25, ARPACI-DUSSEAU

作业（编程）

在这个作业中，你要熟悉一下刚刚读到的进程管理（process management）API。别担心，它比听起来还要有趣！一般来说，你越能抽出时间写代码，收获就越大，那为什么不现在就开始呢？

问题

1. 编写一个调用 `fork()` 的程序。在调用 `fork()` 之前，让主进程（main process）访问一个变量（例如 x），并把它设为某个值（例如 100）。子进程中这个变量的值是多少？当子进程和父进程都修改 x 的值时，这个变量会怎样？
2. 编写一个程序，它先（用 `open()` 系统调用）打开一个文件，然后调用 `fork()` 创建新进程。子进程和父进程都能访问 `open()` 返回的文件描述符吗？当它们并发地（concurrently）写这个文件，也就是同时写入时，会发生什么？
3. 再用 `fork()` 编写一个程序。子进程应该打印“hello”，父进程应该打印“goodbye”。你应该设法确保子进程总是先打印；你能不在父进程中调用 **`wait()`** 就做到这一点吗？
4. 编写一个程序，调用 `fork()`，然后调用某种形式的 `exec()` 来运行程序 `/bin/ls`。看看你能否试遍 `exec()` 的所有变体，包括（在 Linux 上的）`execl()`、`execle()`、`execlp()`、`execv()`、`execvp()` 和 `execvpe()`。你认为同一个基本调用为什么会有这么多变体？
5. 现在编写一个程序，在父进程中用 `wait()` 等待子进程结束。`wait()` 返回什么？如果在子进程中使用 `wait()` 会发生什么？
6. 对上一个程序稍作修改，这次用 `waitpid()` 代替 `wait()`。什么时候 `waitpid()` 会有用？
7. 编写一个程序，创建一个子进程，然后在子进程中关闭标准输出。如果子进程关闭描述符之后调用 `printf()` 打印一些输出，会发生什么？
8. 编写一个程序，创建两个子进程，并用 `pipe()` 系统调用把一个子进程的标准输出连接到另一个子进程的标准输入（standard input）。

 © 2008–25, ARPACI-DUSSEAU